SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security

COURSE CODE: CSA5112

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO* (BL4 , BL5)

Assessment Tool Weightage: 40%

QUESTIONS: 5 **TOTAL MARKS: 25**

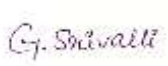
EXPERIMENTS

Code of Conduct:

I _G. Srivalli (192365014)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

1. Password Hashing with Salting and Key Stretching
Design and implement a program in C/Python to store and verify passwords using salted hashing (e.g., bcrypt or PBKDF2). Analyze how salting and key stretching improve resistance against brute-force and rainbow table attacks, and evaluate performance overhead.

---

2. Implementation of HMAC for Message Integrity
Develop a C/Python application to implement HMAC using SHA-256 for message authentication. Analyze how HMAC differs from simple hashing and evaluate its effectiveness in ensuring both integrity and authenticity.

---

3. Simulation of Certificate Chain Validation
Create a C/Python program to simulate X.509 certificate chain verification. Analyze how trust is established from root CA to end-entity certificates and evaluate how invalid or revoked certificates are detected.

---

4. Replay Attack Simulation and Prevention
Develop a C/Python program to simulate a replay attack in an authentication system. Implement a prevention mechanism using nonces or timestamps. Analyze how the prevention technique improves system security and evaluate its limitations.

---

5. Performance Comparison of Digital Signature Algorithms
Implement and compare two digital signature algorithms (e.g., RSA and DSA/ECDSA) in C/Python. Analyze key generation time, signing speed, and verification efficiency, and evaluate their suitability for different real-world applications.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

1. Password Hashing with Salting and Key Stretching

Question

Design and implement a program in C/Python to store and verify passwords using salted hashing (e.g., bcrypt or PBKDF2). Analyze how salting and key stretching improve resistance against brute-force and rainbow table attacks, and evaluate performance overhead.

Aim

To securely store passwords using a unique salt and PBKDF2 key stretching, and to verify a supplied password without storing the original password.

Concept

A password should not be stored directly because a database compromise would immediately reveal users' credentials. Instead, a password is transformed into a derived value using a password-based key derivation function (KDF). A salt is a random value stored with the derived password. The same password therefore produces different stored values for different users or password records. Key stretching repeats the derivation operation many times, deliberately increasing the computation required for every password guess.

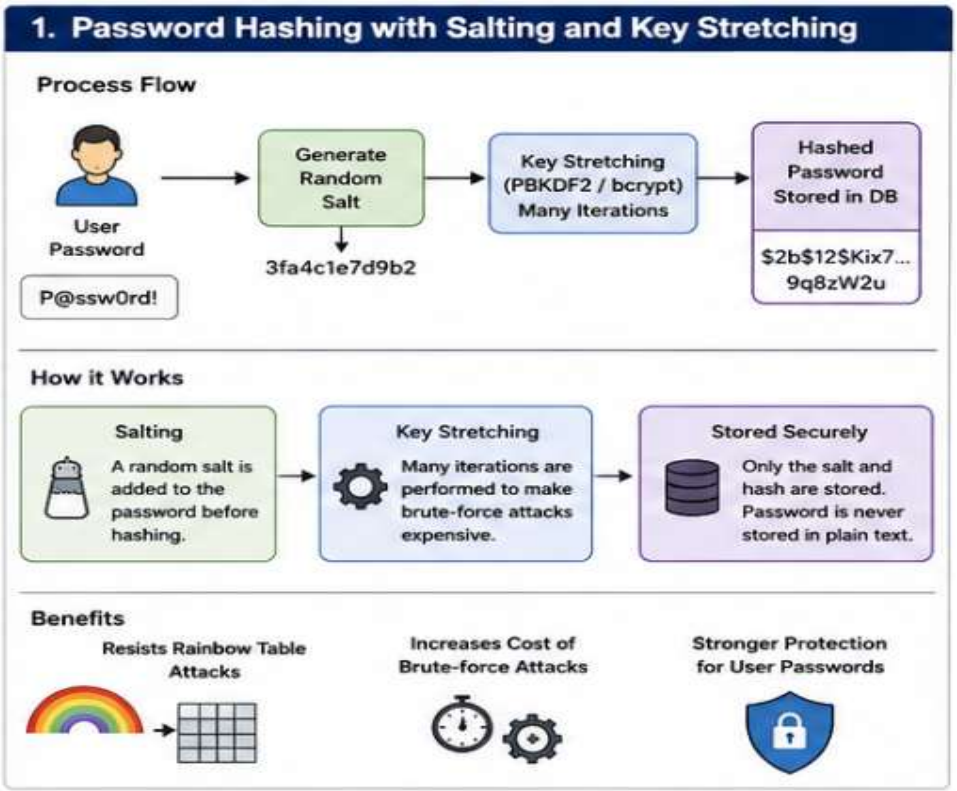


Figure 1: Password storage and verification using salt + PBKDF2

Detailed Explanation

- Salting: A cryptographically random salt is generated for each password. The salt does not need to be secret; it prevents identical passwords from having identical stored hashes.
- Rainbow-table resistance: Precomputed tables depend on reusing the same password-to-hash mapping. A unique salt changes the input to the KDF, making a single precomputed table ineffective across many users.
- Key stretching: PBKDF2 performs a large number of pseudorandom-function iterations. An attacker who makes millions of guesses must pay approximately the same computational cost for each guess.
- Brute-force resistance: Salting does not make a weak password strong. It mainly prevents precomputation and hash reuse. Key stretching increases the cost of each candidate password, slowing large-scale guessing.
- Verification: The server retrieves the stored salt and iteration count, derives a key from the entered password, and compares the derived value with the stored value using a safe comparison operation.

Python Implementation

```
import os
import hashlib
import hmac
import base64
ITERATIONS = 600_000
SALT_SIZE = 16
KEY_SIZE = 32
def hash_password(password):
    salt = os.urandom(SALT_SIZE)
    dk = hashlib.pbkdf2_hmac(
        "sha256", password.encode(), salt, ITERATIONS, dklen=KEY_SIZE
    )
    return {
        "algorithm": "PBKDF2-HMAC-SHA256",
        "iterations": ITERATIONS,
        "salt": base64.b64encode(salt).decode(),
        "hash": base64.b64encode(dk).decode()
    }
def verify_password(password, record):
    salt = base64.b64decode(record["salt"])
    expected = base64.b64decode(record["hash"])
    actual = hashlib.pbkdf2_hmac(
        "sha256", password.encode(), salt,
        record["iterations"], dklen=len(expected)
```

```
    )
    return hmac.compare_digest(actual, expected)
record = hash_password("College@123")
print("Stored record:", record)
print("Correct password:", verify_password("College@123", record))
print("Wrong password:", verify_password("WrongPassword", record))
```

Illustrative Output

```
Stored record: {'algorithm': 'PBKDF2-HMAC-SHA256', 'iterations': 600000,
'salt': '<random Base64 salt>', 'hash': '<derived key>'}
Correct password: True
Wrong password: False
```

Performance and Security Analysis

Technique	Security benefit	Performance effect
Plain hash only	Fast to attack; poor password-storage design	Very low cost
Salted hash/KDF	Defeats simple precomputed/rainbow-table reuse	Small storage overhead; hashing cost depends on KDF
PBKDF2 with high iterations	Makes each password guess expensive	Higher CPU time per login/verification

The exact runtime depends on the CPU, Python version, iteration count, and system load. Therefore, a performance measurement should be taken on the target machine rather than assuming a universal time. In production, the iteration count should be selected by benchmarking so that legitimate authentication remains usable while each offline password guess is expensive.

Conclusion

Salted password hashing with key stretching provides substantially better protection than storing passwords or using a fast unsalted hash. The main trade-off is additional computation during password creation and verification, which is intentional in secure password storage.

2. Implementation of HMAC for Message Integrity

Question

Develop a C/Python application to implement HMAC using SHA-256 for message authentication. Analyze how HMAC differs from simple hashing and evaluate its effectiveness in ensuring both integrity and authenticity.

Aim

To generate and verify an HMAC-SHA-256 tag so that a receiver can detect message modification and verify that the sender possessed the shared secret key.

Concept

A cryptographic hash such as SHA-256 produces a fixed-length digest from a message, but the digest alone does not prove who generated it. Anyone who changes a message can calculate a new ordinary hash. HMAC combines a secret key with a cryptographic hash construction, so an attacker without the key cannot normally create a valid tag for a modified message.

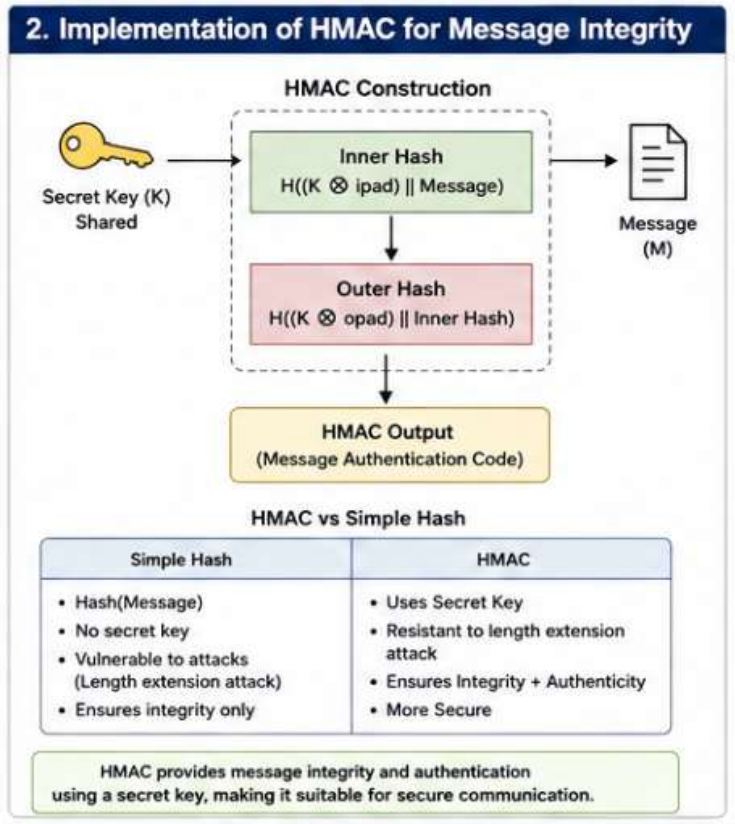


Figure 2: HMAC generation and verification

Python Implementation

```
import hmac
import hashlib
def create_hmac(message, key):
    return hmac.new(
        key.encode(),
        message.encode(),
        hashlib.sha256
    ).hexdigest()
def verify_hmac(message, key, received_tag):
    expected = create_hmac(message, key)
    return hmac.compare_digest(expected, received_tag)
message = "Transfer ₹5000 to account 1234"
key = "shared-secret-key"
tag = create_hmac(message, key)
print("HMAC:", tag)
print("Valid message:", verify_hmac(message, key, tag))
tampered = "Transfer ₹9000 to account 1234"
print("Tampered message:", verify_hmac(tampered, key, tag))
```

Illustrative Output

```
HMAC: <64 hexadecimal characters>
Valid message: True
Tampered message: False
```

HMAC vs Simple Hashing

Property	SHA-256 hash	HMAC-SHA-256
Secret key required	No	Yes
Detects accidental/intentional modification	Yes, if trusted digest is available separately	Yes, when the secret key is protected
Authenticates possession of a shared secret	No	Yes
Suitable for shared-key message authentication	No	Yes

Analysis

HMAC provides message integrity and authentication in a symmetric-key setting. Its security depends on protecting the shared secret key and using a secure hash function. HMAC does not by itself provide non-repudiation because both communicating parties know the same secret key. If a system requires a publicly verifiable signature and stronger non-repudiation properties, a digital signature scheme is more appropriate.

Conclusion

HMAC-SHA-256 is an efficient and widely used mechanism for authenticating messages when communicating parties share a secret key. A simple hash alone cannot provide the same authentication property because it does not depend on a secret.

3. Simulation of Certificate Chain Validation

Question

Create a C/Python program to simulate X.509 certificate chain verification. Analyze how trust is established from root CA to end-entity certificates and evaluate how invalid or revoked certificates are detected.

Aim

To simulate the logical checks performed when validating a certificate chain from a trusted root CA through an intermediate CA to an end-entity certificate.

Concept

X.509 certificates bind an identity to a public key through a certificate signed by a certification authority (CA). A typical chain is End Entity → Intermediate CA → Root CA. The root CA is trusted directly by the relying system. The intermediate certificate is trusted because it is signed by a trusted issuer, and the end-entity certificate is trusted when its issuer, signature, validity period, key usage, name constraints and other applicable checks are valid.

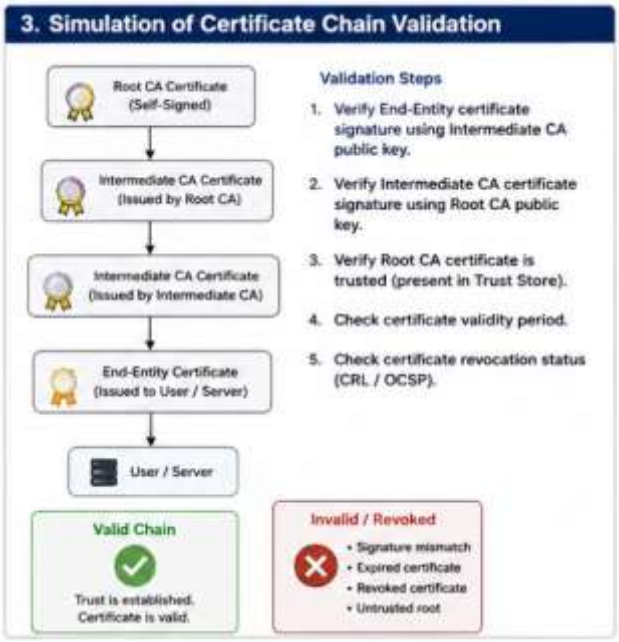


Figure 3: Simplified X.509 certificate chain

Python Simulation

```
from datetime import date
class Cert:
    def __init__(self, subject, issuer, valid_from, valid_to,
                 signature_valid=True, revoked=False, ca=False):
        self.subject = subject
```

```
        self.issuer = issuer
        self.valid_from = valid_from
        self.valid_to = valid_to
        self.signature_valid = signature_valid
        self.revoked = revoked
        self.ca = ca
def valid_now(cert, today):
    return cert.valid_from <= today <= cert.valid_to
def verify_chain(root, intermediate, end_entity, today):
    if root.issuer != root.subject or not root.ca:
        return False, "Root is not a valid trust anchor"
    if intermediate.issuer != root.subject or not intermediate.ca:
        return False, "Invalid intermediate issuer/CA relationship"

    if end_entity.issuer != intermediate.subject:
        return False, "Invalid end-entity issuer"
    for cert in [root, intermediate, end_entity]:
        if not valid_now(cert, today):
            return False, f"{cert.subject}: certificate expired/not yet
valid"
        if not cert.signature_valid:
            return False, f"{cert.subject}: invalid signature"
        if cert.revoked:
            return False, f"{cert.subject}: certificate revoked"
    return True, "Certificate chain is valid"
today = date(2026, 8, 9)
root = Cert("Root CA", "Root CA", date(2020,1,1), date(2035,1,1),
ca=True)
intermediate = Cert("Intermediate CA", "Root CA", date(2024,1,1),
date(2030,1,1), ca=True)
server = Cert("www.example.edu", "Intermediate CA",
              date(2026,1,1), date(2027,1,1))
print(verify_chain(root, intermediate, server, today))
server.revoked = True
print(verify_chain(root, intermediate, server, today))
```

Illustrative Output

```
(True, 'Certificate chain is valid')
(False, 'www.example.edu: certificate revoked')
```

How Trust Is Established

1. The verifier begins with a configured trusted root CA, which acts as the trust anchor.
2. The intermediate CA certificate is checked to ensure that its issuer is the trusted root and that the certificate is authorized to act as a CA.
3. The end-entity certificate is checked to ensure that its issuer matches the expected intermediate CA.
4. Each certificate is checked for signature validity, validity period, and applicable certificate constraints.
5. The verifier checks revocation status using mechanisms such as CRLs or OCSP where applicable.
6. If all required checks succeed and the presented identity matches the certificate's intended name, the chain can be accepted.

Invalid and Revoked Certificate Detection

- Invalid signature: indicates that the certificate was not correctly signed by the claimed issuer or has been altered.
- Expired/not-yet-valid certificate: detected by checking the certificate validity interval against the current time.
- Wrong issuer or broken chain: detected when issuer/subject relationships do not form a valid path to a trusted root.
- Revocation: checked through certificate-status mechanisms such as CRLs and OCSP. A revoked certificate should not be trusted even if its signature and dates are otherwise valid.
- Name mismatch: for TLS-style use, the requested host name must match the certificate's permitted identity fields.

Conclusion

Certificate-chain validation creates a path of cryptographic trust from a configured root CA to an end entity. Security depends not only on signatures but also on validity dates, constraints, identity matching, and appropriate revocation/status checking.

4. Replay Attack Simulation and Prevention

Question

Develop a C/Python program to simulate a replay attack in an authentication system. Implement a prevention mechanism using nonces or timestamps. Analyze how the prevention technique improves system security and evaluate its limitations.

Aim

To demonstrate how an attacker can replay a previously valid authentication message and then prevent the replay by requiring a fresh nonce.

Concept

A replay attack occurs when an attacker captures a valid message and later transmits the same message again. Even if the attacker cannot decrypt or modify the message, replaying a valid authentication token can cause the server to repeat an operation or accept an old authentication attempt. A nonce is a fresh, unpredictable value used once. If the server requires the client to authenticate a newly issued nonce, a previously captured response will not be valid for a later challenge.

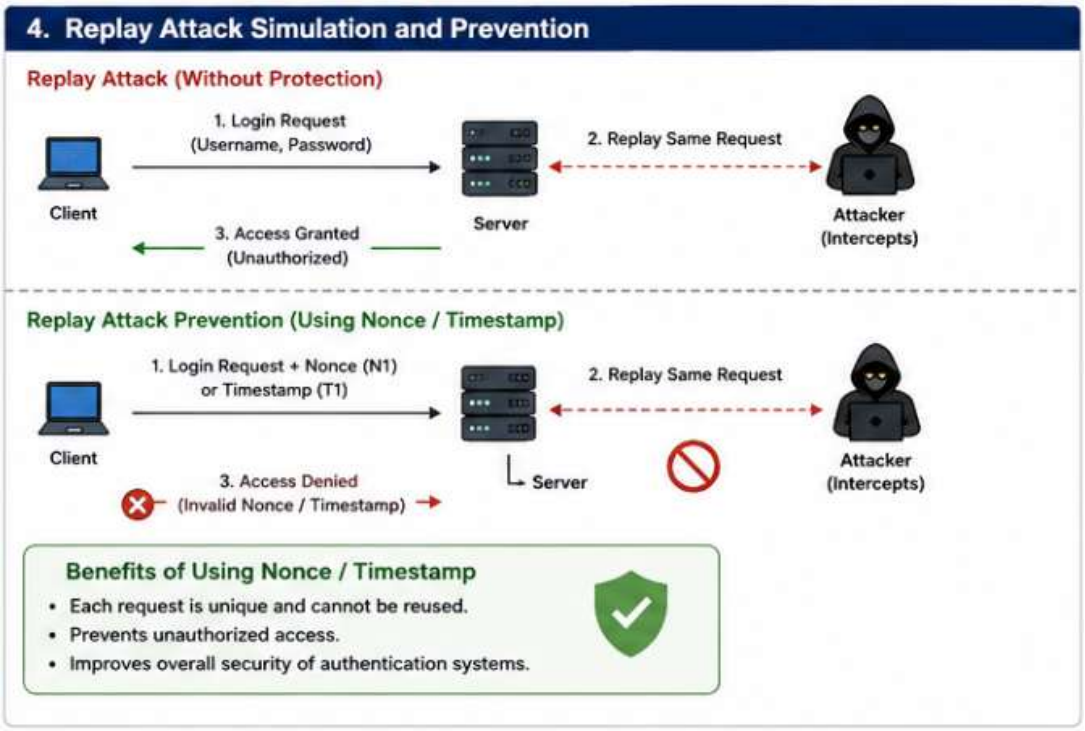


Figure 4: Challenge–response protection against replay

Python Simulation

```
import secrets
import hmac
import hashlib
KEY = b"shared-secret"
def make_response(nonce, user):
    msg = user.encode() + b"|" + nonce
    return hmac.new(KEY, msg, hashlib.sha256).hexdigest()
# Normal authentication
user = "student01"
nonce1 = secrets.token_bytes(16)
response1 = make_response(nonce1, user)
used_nonces = {nonce1}
# A replay attempt reuses the old nonce/response
def server_verify(user, nonce, response):
    if nonce in used_nonces:
        # For a real server, the nonce would be marked used
        # only after successful verification and then removed/expired.
        pass

    expected = make_response(nonce, user)
    if hmac.compare_digest(expected, response):
        return True
    return False
print("Original response valid:", server_verify(user, nonce1,
response1))
# Simulate a fresh challenge: old response is bound to nonce1,
# but the server expects a new nonce2.
nonce2 = secrets.token_bytes(16)
print("Replay against fresh nonce:",
      hmac.compare_digest(make_response(nonce2, user), response1))
```

Illustrative Output

```
Original response valid: True
Replay against fresh nonce: False
```

How the Prevention Works

1. The server creates a cryptographically random challenge nonce for a new authentication attempt.
2. The client creates an authentication response that cryptographically binds the nonce to the user identity.

3. The server verifies the response using the shared secret and confirms that the nonce belongs to the current session.
4. A nonce must not be accepted again. It should be tracked until consumed or allowed to expire.
5. A captured response from an earlier session cannot be reused successfully because it is bound to a different challenge.

Analysis and Limitations

Benefit	Limitation / consideration
Freshness prevents reuse of old authentication messages.	The nonce must be unpredictable and sufficiently long.
No need to rely only on a narrow time window.	The server must store or track outstanding/used nonces.
Works well with MACs and digital signatures.	A timestamp approach requires synchronized clocks and an acceptable clock-skew window.
Reduces replay risk without hiding the protocol message.	Replay prevention does not by itself solve stolen credentials, endpoint compromise, or all session-hijacking attacks.

Conclusion

Replay attacks exploit the reuse of a previously valid message. Binding authentication to a fresh nonce provides a strong freshness check. In practice, nonce generation, storage, expiration, and protocol state must all be implemented carefully.

5. Performance Comparison of Digital Signature Algorithms

Question

Implement and compare two digital signature algorithms (e.g., RSA and DSA/ECDSA) in C/Python. Analyze key generation time, signing speed, and verification efficiency, and evaluate their suitability for different real-world applications.

Aim

To implement RSA and ECDSA signatures in Python, measure the time required for key generation, signing, and verification, and interpret the results for practical use.

Concept

A digital signature provides integrity and public-key authentication: a signer uses a private key to create a signature and a verifier uses the corresponding public key to verify it. Unlike HMAC, the verifier does not need the signer's private key. This property supports public verification and is important for certificates, software updates, documents, and secure protocols.

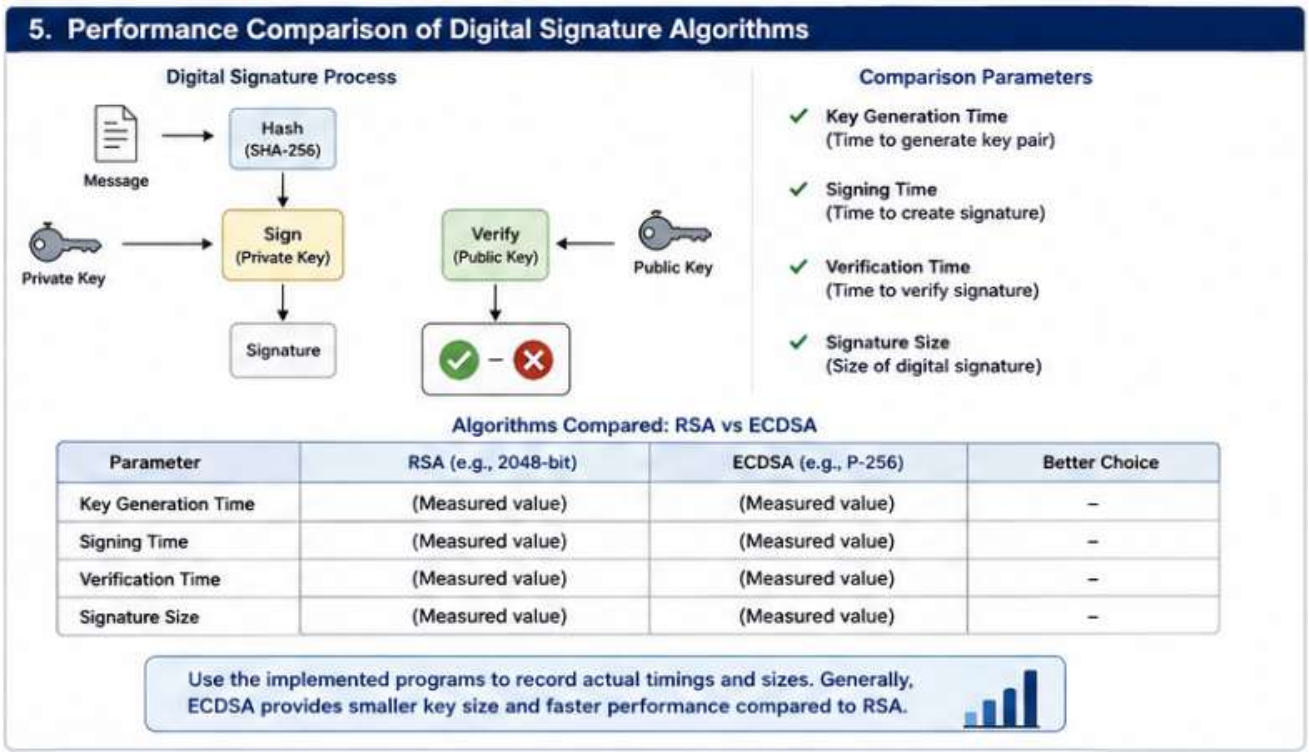


Figure 5: Digital signature workflow

Python Implementation

```
# Requires: pip install cryptography
import time
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import rsa, ec, padding
from cryptography.hazmat.primitives.asymmetric.utils import Prehashed
message = b"Cryptography and Network Security - AT1"
def benchmark_rsa():
    t0 = time.perf_counter()
    private_key = rsa.generate_private_key(
        public_exponent=65537, key_size=2048
    )
    keygen = time.perf_counter() - t0
    t0 = time.perf_counter()
    signature = private_key.sign(
        message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )
    signing = time.perf_counter() - t0
    t0 = time.perf_counter()
    private_key.public_key().verify(
        signature, message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )
    verification = time.perf_counter() - t0
    return keygen, signing, verification, len(signature)
def benchmark_ecdsa():
    t0 = time.perf_counter()
    private_key = ec.generate_private_key(ec.SECP256R1())
    keygen = time.perf_counter() - t0
    t0 = time.perf_counter()
    signature = private_key.sign(message, ec.ECDSA(hashes.SHA256()))
    signing = time.perf_counter() - t0
```

```
    t0 = time.perf_counter()
    private_key.public_key().verify(
        signature, message, ec.ECDSA(hashes.SHA256())
    )
    verification = time.perf_counter() - t0
    return keygen, signing, verification, len(signature)
print("RSA:", benchmark_rsa())
print("ECDSA:", benchmark_ecdsa())
```

Illustrative Result Format

The following table is a result template rather than a claim of universal benchmark values. Run the program on the target computer and replace the placeholders with measured times. Hardware, operating system, Python version, cryptography backend, CPU load, and implementation details can change the measurements.

Algorithm	Key generation	Signing	Verification
RSA-2048	Measured value	Measured value	Measured value
ECDSA P-256	Measured value	Measured value	Measured value

Comparison and Analysis

Factor	RSA	ECDSA	Practical implication
Key size / signature size	Larger keys and signatures	Much smaller keys/signatures at comparable security levels	ECDSA is attractive where bandwidth and storage are constrained.
Signing	Typically efficient but depends on key size and padding	Efficient with elliptic-curve operations	Both are practical; benchmark the actual platform.
Verification	Often fast with common public exponent	Also efficient, with different performance characteristics	Application workload determines the better choice.
Deployment ecosystem	Very broad and mature	Widely deployed in modern TLS, mobile and embedded systems	Compatibility and library support matter.

Suitability for Real-World Applications

- RSA is a strong general-purpose choice where broad compatibility, established PKI infrastructure, or legacy support is important. Modern RSA deployments should use adequate key sizes and secure signature padding such as RSA-PSS.
- ECDSA is useful when smaller public keys and signatures are valuable, such as constrained devices, mobile systems, and protocols where reducing transmitted data matters.
- For new systems, the algorithm should be selected based on current standards, library support, security requirements, interoperability, and measured performance rather than benchmark results from a different machine.
- A digital-signature implementation should use a well-maintained cryptographic library rather than implementing the underlying mathematical primitives from scratch.

Conclusion

RSA and ECDSA both provide digital signatures, but they have different key sizes, performance characteristics, and ecosystem considerations. A meaningful experiment should measure both algorithms on the same machine under the same conditions and report the actual observations.